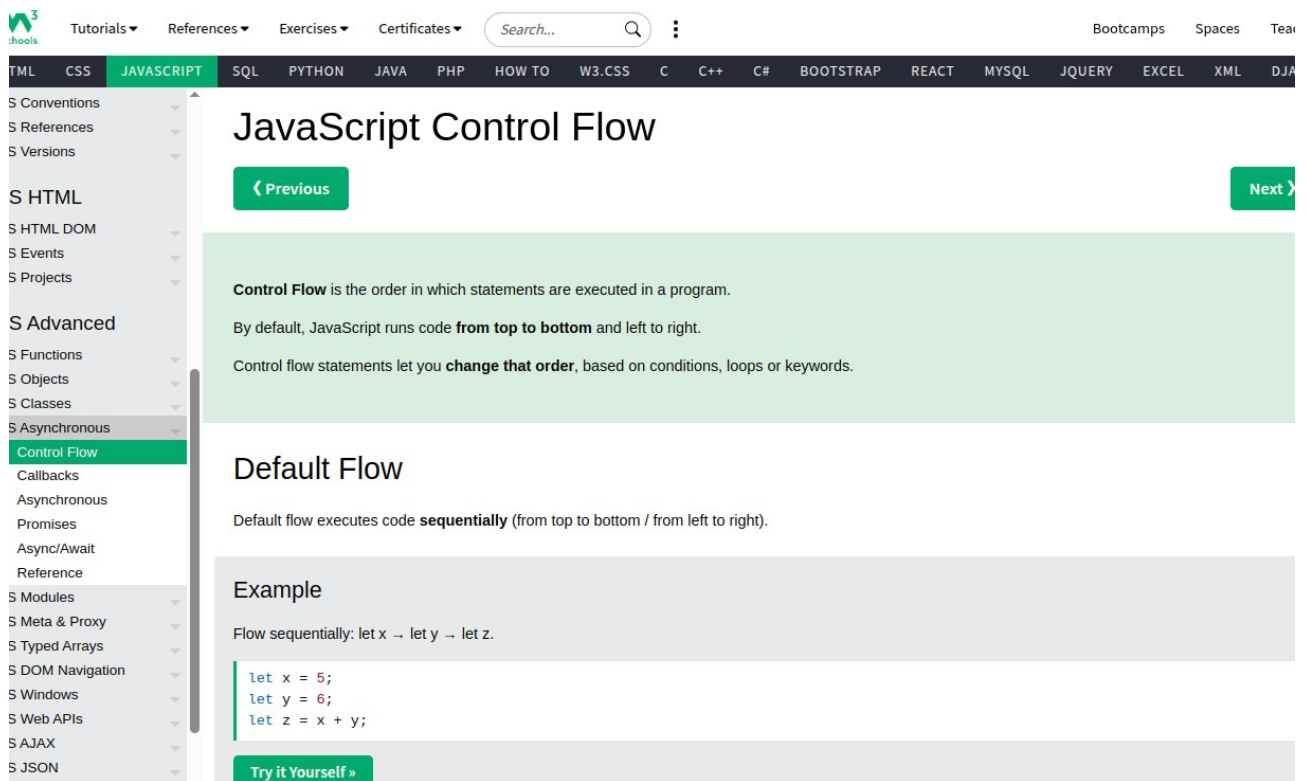


Actividad 5-19: Capturas - Asynchronous

Control Flow:



The screenshot shows the w3schools website's 'JavaScript Control Flow' page. The left sidebar contains a navigation menu with categories like 'Tutorials', 'References', 'Exercises', and 'Certificates'. Under 'JavaScript', there are sub-menus for 'Conventions', 'References', 'Versions', 'HTML', 'HTML DOM', 'Events', 'Projects', 'Advanced', 'Functions', 'Objects', 'Classes', 'Asynchronous', 'Control Flow' (highlighted), 'Callbacks', 'Asynchronous', 'Promises', 'Async/Await', 'Reference', 'Modules', 'Meta & Proxy', 'Typed Arrays', 'DOM Navigation', 'Windows', 'Web APIs', 'AJAX', and 'JSON'. The main content area is titled 'JavaScript Control Flow' and includes a 'Previous' button and a 'Next' button. The text explains that 'Control Flow' is the order in which statements are executed in a program. By default, JavaScript runs code from top to bottom and left to right. Control flow statements let you change that order, based on conditions, loops or keywords. The 'Default Flow' section states that default flow executes code sequentially (from top to bottom / from left to right). An 'Example' section shows a code snippet:

```
let x = 5;
let y = 6;
let z = x + y;
```

 and a 'Try it Yourself' button.

Callbacks:

HTML CSS **JAVASCRIPT** SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

JS References
JS Versions
JS HTML
JS HTML DOM
JS Events
JS Projects
JS Advanced
JS Functions
JS Objects
JS Classes
JS Asynchronous
Control Flow
Callbacks
Asynchronous
Promises
Async/Await
Reference
JS Modules
JS Meta & Proxy
JS Typed Arrays
JS DOM Navigation
JS Windows
JS Web APIs
JS AJAX
JS JSON
JS jQuery
JS Graphics

JavaScript Callbacks

[< Previous](#) [Next >](#)

"I will call back later!"

A callback is a function passed as an argument to another function

This technique allows a function to call another function

A callback function can run after another function has finished

Function Sequence

JavaScript functions are executed in the sequence they are called. Not in the sequence they are defined.

This example will display "Hello", but end up with "Goodbye" because the functions are called in that order:

Example

```
function myFirst() {  
  myDisplayer("Hello");  
}  
  
function mySecond() {  
  myDisplayer("Goodbye");  
}
```

Asynchronous:

HTML CSS **JAVASCRIPT** SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

JS Versions
JS HTML
JS HTML DOM
JS Events
JS Projects
JS Advanced
JS Functions
JS Objects
JS Classes
JS Asynchronous
Control Flow
Callbacks
Asynchronous
Promises
Async/Await
Reference
JS Modules
JS Meta & Proxy
JS Typed Arrays
JS DOM Navigation
JS Windows
JS Web APIs
JS AJAX
JS JSON
JS jQuery
JS Graphics
JS Examples

Asynchronous JavaScript

[< Previous](#) [Next >](#)

"I will finish later!"

Functions running in **parallel** with other functions are called **asynchronous**

A good example is JavaScript setTimeout()

Asynchronous JavaScript

The examples used in the previous chapter, was very simplified.

The purpose of the examples was to demonstrate the syntax of callback functions:

Example

```
function myDisplayer(something) {  
  document.getElementById("demo").innerHTML = something;  
}  
  
function myCalculator(num1, num2, myCallback) {  
  let sum = num1 + num2;  
  myCallback(sum);  
}
```

Promises:

schools

HTML CSS **JAVASCRIPT** SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

JS HTML
JS HTML DOM
JS Events
JS Projects

JS Advanced
JS Functions
JS Objects
JS Classes
JS Asynchronous
Control Flow
Callbacks
Asynchronous
Promises
Async/Await
Reference

JS Modules
JS Meta & Proxy
JS Typed Arrays
JS DOM Navigation
JS Windows
JS Web APIs
JS AJAX
JS JSON
JS jQuery
JS Graphics
JS Examples
JS Reference

JavaScript Promises

[< Previous](#) [Next >](#)

"I Promise a Result!"

"Producing code" is code that can take some time
"Consuming code" is code that must wait for the result
A Promise is an Object that links Producing code and Consuming code

The **Promise Object** represents the completion or failure of an asynchronous operation and its results.
A Promise can have 3 states:

pending	initial state
rejected	operation failed
fulfilled	operation completed

Async/Await:

schools

HTML CSS **JAVASCRIPT** SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

JS HTML
JS HTML DOM
JS Events
JS Projects

JS Advanced
JS Functions
JS Objects
JS Classes
JS Asynchronous
Control Flow
Callbacks
Asynchronous
Promises
Async/Await
Reference

JS Modules
JS Meta & Proxy
JS Typed Arrays
JS DOM Navigation
JS Windows
JS Web APIs
JS AJAX
JS JSON
JS jQuery
JS Graphics
JS Examples
JS Reference

JavaScript Async

[< Previous](#) [Next >](#)

"async and await make promises easier to write"

async makes a function return a Promise
await makes a function wait for a Promise

Async Syntax

The keyword **async** before a function makes the function return a promise:

Example

```
async function myFunction() {
  return "Hello";
}
```

Is the same as:

```
function myFunction() {
  return Promise.resolve("Hello");
}
```

Reference:

schools

HTML CSS JAVASCRIPT SQL PYTHON JAVA PHP HOW TO W3.CSS C C++ C# BOOTSTRAP REACT MYSQL JQUERY EXCEL XML DJANGO

JS HTML DOM
JS Events
JS Projects

JS Advanced
JS Functions
JS Objects
JS Classes
JS Asynchronous
Control Flow
Callbacks
Asynchronous
Promises
Async/Await
Reference
JS Modules
JS Meta & Proxy
JS Typed Arrays
JS DOM Navigation
JS Windows
JS Web APIs
JS AJAX
JS JSON
JS jQuery
JS Graphics
JS Examples
JS Reference

JavaScript Promise Reference

[< Previous](#)

[Next >](#)

The **Promise Object** represents the completion or failure of an asynchronous operation and its results.

A Promise can have 3 states:

pending	initial state
rejected	operation failed
fulfilled	operation completed

Promise Object Methods & Properties

Revised December 2025

Name	Description
<code>catch()</code>	Provides a function to be called when a promise is rejected
<code>finally()</code>	Provides a function to be called when a promise is fulfilled or rejected
<code>then()</code>	Provide two functions to be called when a promise is fulfilled or rejected